Realtime CME Pipeline Documentation

Gergely Koban

July 2026

1 Introduction

This document describes the scripts, workflows, and recovery mechanisms used by the real-time SOFIE pipeline, which automatically initializes and executes CME–SEP simulations using event information obtained from the DONKI database.

2 Overview

This documentation describes the automated SOFIE space weather forecasting pipeline developed at the University of Michigan for real-time modeling of the ambient solar wind, coronal mass ejections (CMEs), and solar energetic particles (SEPs). The purpose of the pipeline is to provide an end-to-end operational workflow that begins with regularly updated solar input observations and produces forecast products that can be displayed through the CLEAR website.

The pipeline is built around the Solar Wind With Field Lines and Energetic Particles (SOFIE) framework, which couples several physics-based models. The ambient solar wind and CME propagation are modeled with the Alfvén Wave Solar atmosphere Model–Realtime (AWSoM-R). CMEs are initiated using the Eruptive Event Generator using Gibson–Low configuration (EEGGL), and SEP acceleration and transport are modeled using M-FLAMPA and MITTENS. Together, these components allow the system to forecast heliospheric solar wind conditions, ICME arrival, and time-dependent SEP proton fluxes at operationally relevant energies.

In the current operational setup, the main simulations are executed on NASA's Pleiades supercomputer. The ambient solar wind run is updated regularly using newly available photospheric magnetic field observations, maintaining an up-to-date heliospheric background solution. When a CME event is detected and initialized, the system branches from the background solar wind state and launches a coupled CME–SEP simulation. This design allows the pipeline to preserve a continuously running background model while also producing event-specific forecasts.

After the simulations are completed on Pleiades, the output files are transferred to 'solsticedisk', a University of Michigan CLaSP disk farm, where the postprocessing workflow is executed. The postprocessing step converts the raw

model output into forecast products suitable for inspection and web display, including solar wind plots, CME/ICME arrival information, SEP proton flux profiles, and other derived products. The CLEAR website then accesses the processed results from 'solsticedisk' and presents them in a user-facing format.

This documentation is intended to describe the structure, execution, data flow, and maintenance of the pipeline. It includes information about the major model components, input data sources, simulation workflow, file transfers, postprocessing steps, website integration, and troubleshooting procedures. The goal is to make the pipeline understandable, reproducible, and maintainable for future users and developers.

3 Realtime Solar Wind Background

3.1 Scripts

3.1.1 job_recurrent_cron_failsafe.sh

This is the main jobscript for the background simulation, a PBS batch-job script designed to run the real-time SWMF/AWSoM solar-wind simulation repeatedly for approximately one day. It requests 13 Cascade Lake computing nodes with 40 processor cores per node, giving a total of 520 MPI processes, and it has a maximum wall time of 25 hours. At the beginning, the script loads the required compiler, MPI, GCC, and Python modules, removes the stack-size limit, and redirects all standard output and error messages to the file `/nobackupp28/gkoban/SWMF_AWSRT/SWMF/output.log`.

An important part of the script is its failsafe and checkpointing system. Before each simulation cycle modifies or removes important files, the script creates a timestamped checkpoint in the `run_realtime/checkpoints` directory. This checkpoint stores the main parameter files, harmonic magnetic-field files, magnetogram timing information, coronal-heating parameters, and available SWMF restart directories. The script also creates a file called `.cycle_in_progress`, which records the current checkpoint, the current processing phase, the time, the host, and the PBS job identifier. If this marker is found at the beginning of a later cycle, the script assumes that the previous cycle did not finish successfully and restores the simulation state from the checkpoint identified by the `last_good` symbolic link.

During each cycle, the current `PARAM.in` file is archived, and the previous end magnetogram time becomes the starting magnetogram time for the new simulation interval. The script then downloads a new magnetogram using `get_magnetogram_pleiades.py`, extracts the downloaded archive, remaps the magnetogram, and runs `HARMONICS.exe` to generate the harmonic representation required by SWMF. If the newly downloaded magnetogram has the same timestamp as the previous one, the script waits for five minutes and tries again. This process continues until a magnetogram with a newer timestamp becomes available.

Once the new magnetogram has been prepared, the script creates a new `PARAM.in` file from the restart configuration and runs the SWMF simulation with the command `mpiexec -n 520 ./SWMF_solar.exe`. The simulation is considered successful only if the MPI command finishes with an exit status of zero and SWMF creates the file `SWMF.SUCCESS`. If either condition is not satisfied, the script exits with an error. It does not restore the checkpoint immediately; instead, it leaves the unfinished-cycle marker in place so that the next execution can detect the failure and restore the most recent successful state.

After a successful simulation, the script runs the post-processing program, combines the individual Earth and STEREO-A satellite output files into `sat_earth.sat` and `sat_sta.sat`, removes the old restart directory, and runs `Restart.pl` to prepare the simulation for the next cycle. Only after all of these steps have completed successfully is the current checkpoint marked as the new `last_good` state. The script then removes older checkpoints while retaining approximately the latest 24 checkpoints.

The nested loops allow the script to perform up to 24 simulation cycles, although the variables representing AM, PM, and the hour are only loop counters and do not force the cycles to occur at specific clock times. At startup, the script also calculates a stopping time 23 hours and 20 minutes in the future. After every completed cycle, it checks whether this time has been reached. If it has, the script performs final post-processing and exits cleanly before the 25-hour PBS wall-time limit. A controlled shutdown can also be requested by creating an `AWSOMRT.STOP` file. Overall, the script repeatedly obtains new solar magnetograms, advances the real-time SWMF simulation, processes the resulting output, and prepares the next restart, while using checkpoints to ensure that an interrupted or failed cycle does not leave the simulation in an unusable state.

3.1.2 get_magnetogram_cron_GONG.py

This script searches the NSO/GONG archive for the most recent available MRZQS magnetogram and saves it as `fitsfile.fits` in the `SUBMISSION_DATA` directory. Rather than checking only the current UTC date, it searches the directory for the following UTC day first, then the current day, and finally a configurable number of previous days. Checking the following day accounts for the fact that GONG observations and directory dates may already have advanced in the earliest time zones, while searching backward provides protection against missing, delayed, or incomplete daily directories.

For every daily directory, the script reads the available filenames and extracts the observation time encoded after the letter `t`. It then selects the file with the latest timestamp instead of relying on the order in which the website lists the files. This is a useful design choice because the ordering of files on a web server is not guaranteed to represent their observation times. The filename-matching expression also accepts two slightly different MRZQS naming formats, making the downloader less sensitive to variations in the GONG archive structure.

The selected magnetogram is downloaded in chunks as a compressed `.fits.gz`

file and is then decompressed to the fixed filename `fitsfile.fits`. Using a fixed local name simplifies the rest of the pipeline, since subsequent scripts do not need to identify the original timestamped GONG filename. The compressed file is normally deleted after extraction, although the `--keep-gz` option allows it to be preserved when necessary.

The script also writes a metadata file named `fitsfile.source.txt`. This file records the download time, source date directory, original filename, source directory, and complete source URL. Before downloading a new magnetogram, the script compares the observation time of the candidate file with the observation time stored in this metadata file. If the candidate is older than or equal to the existing magnetogram, the download is skipped and the current FITS file is retained. This prevents the real-time pipeline from accidentally moving backward in time because of delayed server updates or an older file temporarily appearing as the newest available product. Overall, the script was designed to prioritize robustness, chronological consistency, and traceability rather than simply downloading the first file returned by the server.

The download procedure was also designed to minimize unnecessary network traffic and memory use. The script first reads only the relatively small HTML directory listings needed to identify the newest available magnetogram; it does not download every candidate file in order to determine which one is latest. After selecting a single file, it compares its timestamp with the metadata of the magnetogram already stored locally. If the candidate is not newer, the script terminates without downloading it, avoiding an unnecessary transfer. When a new file is required, the compressed `.fits.gz` product is downloaded using a streaming request and is written to disk in approximately 1MB chunks. This means that the complete compressed file does not need to be held in memory, which is advantageous for an automated process that may run repeatedly and unattended. Transferring the compressed GONG product also requires less bandwidth than transferring an uncompressed FITS file. After decompression, the compressed copy is deleted by default to reduce disk usage, although it can be retained using the `--keep-gz` option.

3.1.3 fitstime_advance.sh

This shell script checks whether the downloaded magnetogram has been updated recently. It first loads the Python environment required by the associated FITS-processing script and verifies that both `fitsfile.fits` and `fitstime_advance.py` exist. It then compares the current system time with the filesystem modification time of the FITS file.

If the magnetogram has not changed for at least 70 minutes, the shell script calls `fitstime_advance.py`. Otherwise, it leaves the file unchanged. The 70-minute threshold provides some tolerance around the expected approximately hourly arrival of new GONG products, so a small publication or transfer delay does not immediately activate the fallback procedure.

3.1.4 fitstime_advance.py

This Python script provides the actual fallback operation used when no new GONG magnetogram has arrived. It opens `fitsfile.fits` in update mode, reads the `DATE` keyword from the primary FITS header, converts it into a Python datetime object, and advances the recorded time by one hour. The updated time is then written back to the same FITS header.

The script checks that the `DATE` keyword exists and that it follows the expected `YYYY-MM-DDTHH:MM` format. If either condition is not satisfied, it stops with an error instead of writing an uncertain value to the file. This is an important safety choice because silently modifying an incorrectly formatted header could produce an incorrect simulation interval.

Advancing the `DATE` header allows the simulation pipeline to reuse the most recently available map with an updated timestamp when the GONG service is temporarily unavailable. This is done in favor of operational continuity, but outputs produced during this fallback period should be interpreted as simulations based on an older magnetic map rather than a genuinely new observation.

3.1.5 get_magnetogram_pleiades.py

This script prepares the contents of `SUBMISSION_DATA` for use by the real-time SWMF job. It removes any temporary directory left by an earlier execution, copies the entire `SUBMISSION_DATA` directory into a new temporary directory under `run_realtime/SC`, and packages the copied files into an archive named `submission.tgz`. After the archive has been created, the temporary directory is deleted.

Although its name refers to obtaining a magnetogram, the script does not download the GONG data itself. Instead, it acts as an interface between the independently updated `SUBMISSION_DATA` directory and the main SWMF simulation. The simulation job can extract the same predictable archive on every cycle, regardless of whether the magnetogram was newly downloaded or temporarily reused by the timestamp-advancement fallback. This design choice is needed since the submitted job itself cannot reach the internet, so the magnetogram download has to be done separately, on a non-compute node.

Copying the source directory before creating the archive is a useful design choice because the archive is assembled from a temporary snapshot rather than directly from files that another cron process might be updating. Removing the temporary directory before and after the operation also reduces the risk that obsolete files from a previous cycle will be included in a new submission archive.

3.1.6 RT_submit_cron.sh

This script supervises the main real-time PBS job. It queries the PBS scheduler for jobs belonging to the current user and checks whether a queued or running job exists with the name `SWMF_MFLAMPA_RT`. If such a job is found, the script records that no action is required. If the job is absent, it submits

`job_recurrent_cron_failsafe.sh` using `qsub` and records the new job identifier.

The result of each check is appended to `recurrent_cron.log` together with an ISO-formatted timestamp. This creates a simple operational history showing when the job was found, when it disappeared, and when a replacement was submitted.

The main advantage of this approach is that the PBS job can recover automatically after reaching its wall-time limit or terminating unexpectedly. At the same time, checking the job name before submission prevents cron from creating multiple copies of the real-time simulation. The supervision mechanism is independent of the simulation itself, so it can restore pipeline operation even when the main job has stopped completely. Together with how the check for abruptly ended runs are being done at the beginning of `job_recurrent_cron_failsafe.sh`, this guarantees that the simulational pipeline can recover after failed simulations.

The script can be run using cron with any frequency, I am running it hourly.

3.1.7 sync_realtime.sh

This script transfers the combined Earth satellite output file, `sat_earth.sat`, from the real-time SWMF run directory on Pleiades to the corresponding results directory on Solstice. The transfer is performed using rsync with archive, partial-transfer, and in-place update options.

The `--partial` option preserves partially transferred data when a connection is interrupted, which can reduce the amount of data that must be retransmitted. The `--inplace` option updates the destination file directly. This is suitable for `sat_earth.sat`, which is repeatedly replaced by a newer version and is expected to remain available under one constant filename.

The script uses `set -euo pipefail`, causing it to stop when a command fails, an unset variable is used, or part of a command pipeline fails. This makes synchronization failures easier to detect and prevents the script from reporting apparent success after an incomplete transfer.

3.1.8 sync_latest_zcut.sh

This script transfers the most recent z=0 output cuts from both the inner-heliospheric and solar-coronal components of the SWMF simulation. It searches the `IH/IO2` and `SC/IO2` directories for files matching the expected `z=0_var__e.out` pattern.

For each component, the script extracts the timestamp located between `_e` and `.out` in the filename. It sorts the matching files by this timestamp and selects the latest one. Choosing the file according to its embedded simulation timestamp is more reliable than using the filesystem modification time, since modification times can change when files are copied, restored, or otherwise processed.

The newest IH file is copied to Solstice as `zcut.out`, while the newest SC file is copied as `zcut_SC.out`. These stable output names are advantageous for visualization and post-processing programs because they can always read the same path instead of searching for a changing timestamped filename. To preserve provenance, the script also creates a separate text file containing the original filename of each transferred output. This allows downstream programs or users to determine the exact simulation time represented by the stable file.

The script checks that suitable files exist and verifies that at least one filename follows the expected timestamp format. It creates the remote destination directory if necessary, preserves the copied file's timestamp using `scp -p`, and changes the remote file permissions after transfer. Strict shell error handling ensures that a failure during one of the transfers stops the script instead of allowing later steps to continue with incomplete data.

4 CME-SEP Simulation

4.1 Parameter files

4.1.1 Param.in.gap

Available online here, this serves as the parameter file for the gaprun. The start time is always the latest magnetogram time, while the endtime is the time of the CME initiation. There is a one minute session before the end with the SP module, which serves to output the `LONLAT.earth` file inside SP/IO2, which is needed for the CME PARAM.in.

It uses *CME_AMR.in*, which governs AMR such that it runs three times during the gap run.

4.1.2 Param.in.CME

Available online here, this serves as the parameter file for the CME run.

4.2 Scripts

4.2.1 CMElaunch_cron.sh

This script is responsible for detecting newly generated CMEs, preparing the corresponding run directory, and submitting the simulation job. At each execution, it scans the directory `/nobackupp28/nbiro/CME_pipeline`, where the EEGGL output is stored. It iterates through all subdirectories to identify CME events that have not yet been processed. New CME folders are determined using two mechanisms: a manifest file that records all previously executed CME events, and a *LAUNCHED* token file placed within directories that have already been submitted.

Whenever a new CME folder is identified, two files are expected inside. The EEGGL output *CME.in*, and the *processed_CME.json*, which contains parts of the DONKI entry for this CME, like CME time. Using these files, a run directory

is created inside the SWMF directory (specified inside the script by the variable SWMF_dir), following the naming convention run_CME_YYYYMMDD_HHmm, where the sequence after run_CME_ is the datetime of the CME. After this, three scripts are called to set up the run directory correctly, and then, the job gets submitted by invoking */PBS/bin/qsub*.

4.2.2 pick_cycle.py

This script is called inside CMElaunch_cron.sh, and is responsible for picking the correct checkpoint from the background runs. The background runs store the last 24 checkpoints, which contain the RESTART files, magnetograms, and everything else needed for the CME run. This script identifies the last one before the CME time and gives back the path to that checkpoint. It works with two command line arguments: `--run-realtime` specifies the path to the realtime background runs, and `--cme-root` specifies the path to the cme folder.

4.2.3 setup_CMEdir.sh

This script is invoked inside CMElaunch_cron.sh, and is responsible for creating and configuring the run directory for a CME simulation. It requires two command-line arguments: `--cycle`, which specifies the appropriate checkpoint directory, and `--rundir`, which defines the path to the new run directory.

The setup procedure follows the same logic as for the realtime background runs. It assumes the existence of a generic run directory named `run` within SWMF, from which the necessary files are copied and linked into the new distribution directory. In the case of CME runs, this process is extended by incorporating files from the specified checkpoint.

The checkpoint files copied into the run directory include:

- `harmonics_bxyz.out`,
- `ENDMAGNETOGRAMTIME.in`,
- `endmagnetogram.dat`,
- `CORONALHEATING.in`,

as well as the full `RESTART_n0000000` directory. Finally, symbolic links are created for `RESTART.in` and for the *restartIN* subdirectories within both SC and IH.

4.2.4 make_jobscript.py

This is the third and last script invoked by *CMElaunch_cron.sh*. It expects four arguments: `--SWMF_dir` specifies the path to the SWMF directory, `--template` is the job script template, in our case, called *launch_cme.sh*, `--run-dir` takes the path to the run directory, and, lastly, `--cme-event-dir` gets the event directory for the CME. This script fills in the template (which is under the

SWMF directory) with the path to the run directory and the event directory, then places it inside the run directory with the name *job_CME.sh*. This is the file that ultimately gets submitted into the queue.

4.3 launch_cme.sh

This is the template for the jobscript that runs the simulations. By default, the requested walltime is 10 hours in the long queue, on 2000 processors, on Aitken CascadeLake nodes. The first step is loading the modules. These modules are loaded:

- comp-intel
- mpi-hpe/mpt
- gcc/9.3
- miniconda3/v4

Loading the miniconda also requires the command:
module use -a /swbuild/analytix/tools/modulefiles

Following this setup, the job script consists of two primary phases: the gap run and the CME run.

The gap run is initialized through four dedicated setup scripts. Once these steps are completed, *ParamConvert.pl* is executed with the `PARAM.in.gap` file to generate the parameter file for the gap simulation. This parameter file makes use of the `#INCLUDE` directive to incorporate the following configuration files: `ENDMAGNETOGRAMTIME.in`, `CMETIME.in`, `STOPTIME.in`, `CME_AMR.in`, `IHCME_AMR.in`, and `AMR_settings.in`. All of these input files are generated by Python scripts invoked within the job script. Once the finalized `PARAM.in` file has been assembled, the gap simulation is executed.

After the gap run has finished, *Restart.pl* is used to set up the resulting restart files for the CME run. There is an additional file needed for the CME parameter file other than the ones discussed before, the `CME.in`, which is now copied from the event directory. Then, the *ParamConvert.pl* is called again, this time, with the PARAM.in.CME file to create the CME parameter file. Finally, the CME run is executed.

4.3.1 make_CMEin.py

This script is responsible for creating the `CME_AMR.in` and `IHCME_AMR.in` files from the `CME.in` in the event directory. It takes two command line arguments: `--infile` which is the path to the `CME.in` file (and should include the filename!), and the `--outdir`, which is the output path. In the current version of the jobscript, the output directory is the SC folder inside the run directory, and the *ParamConvert.pl* is called from there. The two resulting files govern the CME-related AMR regions in the SC and IH domains. For the `CME_AMR.in`, we

calculate the LongMin-LongMax and LatMin-LatMax variable pairs based on the following empirical formula:

$$\text{LongMin} = \text{mod}\left(\lambda_{\text{CME}} - 40R_{\text{CME}}, 360\right), \quad \text{LongMax} = \text{mod}\left(\lambda_{\text{CME}} + 40R_{\text{CME}}, 360\right), \tag{1}$$

$$\text{LatMin} = \phi_{\text{CME}} - 20R_{\text{CME}}, \qquad \text{LatMax} = \phi_{\text{CME}} + 20R_{\text{CME}}. \tag{2}$$

For the `IHCME_AMR.in`, the variables yrotate and zrotate change according to this scheme:

$$y_{\text{rotate}} = -\phi_{\text{CME}}, \tag{3}$$

$$z_{\text{rotate}} = \lambda_{\text{CME}}. \tag{4}$$

All other variables are hardwired and do not change.

4.3.2 make_CMEtime.py

This script reads the `processed_CME.json` file from the event directory and generates the corresponding `CMETIME.in` file, formatted according to the requirements of the parameter file. The resulting time definition is included in both the gap-run and CME-run `PARAM.in` files, where it serves as the stop time for the gap simulation and the start time for the subsequent CME simulation. It expects two arguments: the `--cme-json` gives the path to the `processed_CME.json`, including the filename; while the `--rundir` sets the path to the run directory.

4.3.3 make_amr_settings.py

This script creates the `AMR_settings.in` file which governs the `#DoAMR` command inside the gaprun. This script is currently set up so that it calculates the time between the end of the last background run and the time of the CME, and creates the `AMR_settings.in` so that during the gap run, the AMR runs exactly 3 times. This can be changed with the optional `--n-amr` argument when calling the script. Otherwise, using the `--run-dir` argument to specify the path to the run directory is enough.

4.3.4 make_stoptime.py

In the current setup, there is a one-minute phase included at the end of the gap-run where SP is run. This is important to obtain the LONLAT.earth file, which enables running MITTENS together with the CME run. This script is needed to create the `STOPTIME.in` which stops the first session (without SP) just one minute before the end of the gap-run, so the second session involving SP can run, and the LONLAT.earth file can be obtained before the start of the CME simulation.

4.3.5 make_injection.py

There is currently a case-by-case change in the `EfficiencyInj` variable in MFLAMPA, based on the CME speed. The variable is scaled based on a reference speed (1500 km/s), with the following formula:

$\eta_{\rm inj} = \exp\left(\frac{v_{\rm CME}-v_{\rm ref}}{v_{\rm ref}}\right)$

This script is responsible for creating `INJECTION.IN` with the correct `EfficiencyInj` value, and `INJECTION.IN` gets included in the PARAM.in

4.3.6 run_watchdog.sh

The `run_watchdog.sh` script monitors an active SWMF simulation and automatically restarts it if the simulation appears to have stopped making progress. The run directory is supplied as the first command-line argument, while the monitoring interval, the time after which the simulation is considered unresponsive, and the maximum number of permitted restarts can optionally be specified. By default, the script checks the simulation every 600 seconds, considers it stalled if its output has not changed for 1200 seconds, and allows at most three restarts. Making these values configurable allows the same watchdog to be used for simulations with different output frequencies and expected runtimes without modifying the script itself.

At startup, the script verifies that the run directory, restart job script, job-ID history file, and PBS commands are all available. It also uses `set -euo pipefail`, which causes the script to stop when a command fails, an undefined variable is accessed, or an error occurs within a command pipeline. The script keeps track of the active PBS job using `jobid_history.txt`. The last non-empty line of this file is interpreted as the current job identifier, while the number of additional entries is used to calculate how many restarts have already occurred. Each newly submitted job identifier is appended to the same file. This provides both a simple persistent state for the watchdog and a history of the jobs associated with the simulation, even across separate executions of the watchdog script. (Note: manually submitted jobs are not automatically added to `jobid_history.txt`)

Before beginning continuous monitoring, the script checks whether `SWMF.SUCCESS` exists. The presence of this file indicates that the simulation completed normally, in which case the watchdog exits without taking any action. It also checks whether the maximum restart count has already been reached. Limiting the number of automatic restarts is an important safety choice because a simulation affected by a persistent configuration, numerical, or input-data error could otherwise be restarted indefinitely while repeatedly consuming computational resources.

During monitoring, the script searches the run directory for files whose names begin with `runlog` and selects the one with the most recent filesystem modification time. The age of this file is then compared with the configured stale threshold. Monitoring the run log provides a simple indication of whether the simulation is continuing to write output and therefore making progress. Se-

lecting the newest matching file is useful for restarted simulations because each execution produces a separate run log, and the watchdog should evaluate the log associated with the most recent run rather than an older one.

If the newest run log has been modified recently, the watchdog waits for the configured checking interval and repeats the test. If no run log is found, it records this condition and continues waiting rather than immediately restarting the job. This conservative behavior avoids terminating a job during an early initialization period before its first run log has been created, although it also means that a job that never creates a run log will not be restarted solely on that basis.

If the run log has not changed for at least the stale threshold, the script treats the simulation as unresponsive. It first attempts to terminate the current PBS job using `qdel`. A failure of `qdel` is recorded as a warning rather than immediately stopping the script, since the job may already have ended or disappeared from the queue. The watchdog then submits `job_CME_restart.sh` using `qsub`, extracts the numerical part of the returned PBS job identifier, and appends it to `jobid_history.txt`. The script exits after submitting the replacement job, so each watchdog execution can produce at most one restart. A newly launched watchdog can then monitor the newly submitted job using the updated job-ID history.

The script records its activity in both a general watchdog log and a log located inside the individual simulation directory. The messages include timestamps, configuration values, current job identifiers, run-log ages, restart decisions, and newly submitted job identifiers. This detailed logging is useful for reconstructing when a simulation stopped progressing and how the automated recovery system responded. In an automatic run, the watchdog is started from inside `CMElaunch_cron.sh`.

5 Crontabs

The automated pipeline is coordinated through the crontab entries shown in Listing 1. These entries initiate the daily simulation and post-processing workflows, synchronize real-time model output, maintain the latest visualization products, check for CME launches and updated GONG magnetograms, and advance the magnetogram timestamp when necessary.

Listing 1: Crontab entries used to operate the automatic real-time and daily-processing pipelines.

```
15 3 * * * /home6/gkoban/Scripts/Daily_runs.csh >> /home6/gkoban/
    Scripts/cron_log.txt 2>&1
15 5 * * * /home6/gkoban/Scripts/PostProc.csh >> /home6/gkoban/
    Scripts/cron_log_PostProc.txt 2>&1
*/10 * * * * /bin/bash /home6/gkoban/Scripts/sync_realtime.sh >> /
    home6/gkoban/Scripts/sync_realtime.log 2>&1
0 */4 * * * /bin/bash /nobackupp28/gkoban/SWMF_AWSRT/SWMF/
    sync_realtime_3D.sh
```

```
*/30 * * * * /home6/gkoban/Scripts/sync_latest_zcut.sh >> /home6/
    gkoban/Scripts/sync_latest_zcut.log 2>&1
0 * * * * /nobackupp28/gkoban/SWMF_AWSRT/SWMF/RT_submit_cron.sh >> /
    nobackupp28/gkoban/SWMF_AWSRT/SWMF/recurrent_cron.log 2>&1
*/10 * * * * /bin/bash /nobackupp28/gkoban/SWMF_AWSRT/SWMF/
    CMElaunch_cron.sh >> /nobackupp28/gkoban/SWMF_AWSRT/SWMF/
    CMElaunch.log 2>&1
*/10 * * * * /swbuild/analytix/tools/miniconda3_220407/bin/python3 /
    nobackupp28/gkoban/SWMF_AWSRT/SWMF/get_magnetogram_cron_GONG.py
    >> /nobackupp28/gkoban/SWMF_AWSRT/SWMF/Magnetogram/cron_log.txt
    2>&1
*/10 * * * * /nobackupp28/gkoban/SWMF_AWSRT/SWMF/fitstime_advance.sh
    >> /nobackupp28/gkoban/SWMF_AWSRT/SWMF/advance_magtime.log 2>&1
```

6 Common Failure Vectors

As with every automated system, there are of course cases when the simulations or the pipeline itself fails for one reason or another. Sometimes, there are steps that can be taken to ensure recovery, whereas sometimes, human input is necessary to correct the errors. This section in the guide serves to record some of the author's observations about factors and events that affect the stability of the pipeline, and what steps were or can be taken to rectify the situation.

6.1 GONG server

As the realtime pipeline ingests the hourly GONG magnetograms, it is easy to see how the accessibility of the GONG FTP server is a key issue. However, I have experienced sometimes long-term issues with the magnetogram downloads, the source of which I have no control over. First and foremost, it is important to mention that there are three instances of the same server: nispdata, gong and gong2. Out of the three, nispdata seems the most stable, and this was confirmed by the sysadmin there. Near mid June, going into early and mid July, there were serious, ongoing problems with the availability of the GONG servers, namely getting 500 internal server error messages across all instances, and the downloads only working sparsely. After emailing the sysadmin, Jon Britanik (jbritanik@nso.edu), he confirmed that the source of the problem is on their end, since they are "experiencing high loads from web scrapers using proxy farms", and having trouble filtering all this traffic out. As a solution, the outgoing IP address from crontabs on PFE25 (129.99.12.100) has been added as an exception to their filters. Although still somewhat unstable, this helped greatly with the stability of the downloads. It is important to note that according to my own experience, the PFEs and their crontabs have separate IP addresses, so this only affects scripts ran from a crontab.

6.2 Pleiades Front Ends

There are many things affecting the automatization of the pipeline relating to PFEs. First and foremost, the crontabs are different accross PFEs, and one has to make sure that they only install their crontabs to one PFE. However, this also means that if there is an issue with that specific PFE, the whole crontab has to be moved to another. In the author's experience, PFE23 and PFE24 are (or were) highly unstable. PFE26 was stable for a long time, then, taken offline for a month, and finally, it seems completely deleted recently. It is important to note that issues regarding PFEs will not frequently warrant an email from NAS saying that the specific PFE is unavailable. I have encountered two types of error: (1) the PFE is not available, it is not running the crontabs, or (2) the PFE is available, but cannot reach the internet. The first type is the more serious one, and after a while, NAS will probably notice and fix it. However, the second type is more "silent failure" type, but causes the pipeline to not download new magnetograms and not to transfer the results to solstice. This type of problem was very common with PFE24, and required emailing to NAS because they did not seem to notice it.

PFE problems typically require human intervention. So far, I haven't been able to come up with an automatization that can detect PFE failure and move the crontabs automatically to a working PFE. One possible solution might be using two PFEs simultaneously, but since that runs every script twice at the same time, it might introduce previously unencountered errors. Needs investigating.

6.3 Hardware-related issues

Two main nodetypes the pipeline uses are Aitken CascadeLake and Electra Broadwell, with the former serving as the main hardware, and the latter as a fallback. We have encountered a number of seemingly hardware related issues which can be expected with a system like this, most of them confirmed through email by NAS Support. Let's go through some of the symptoms experienced before:

- The job gets put on hold or back into the queue immediately after launch. Since in the jobscript, it is specified to send a user an email when the job starts, this can also manifest in getting multiple emails about the same job starting in case the job gets put back into the queue. Another related symptom might be having multiple jobs on hold without putting them on hold manually. This is most likely caused by one or more erroneous nodes. This was a long-standing issue back in March and was encountered again for a few days between July 23-26.

- Sometimes, the jobs can fail and leave behind a core.numbers file, which can be analyzed by using gdb. By analyzing these files, this issue seems to be related to writing functions and the exact cause is segmentation fault almost all the time. The likely culprit (mentioned in most of the gdb

backtrace file) is the function `ih_modwriteplot_mp_write_plot_` . This problem was mostly encountered in the case of the CME-SEP simulation, however, for a 6 day period, the realtime background simulations also ran into this exact issue. The background simulation stopped with this error 3 times, on July 9, July 12 and July 14, while running on Electra Broadwell nodes. Switching back to CascadeLake Aitken seems to have solved the problem, however, the CME-SEP simulations ran into this problem before on CascadeLake Aitken nodes too. There was no other change in the background pipeline for approximately a month before the segmentation faults started happening. The situation requires more intensive investigation.

- There are instances where qstat shows the job as running, but there is no output in either IO2 folders or in the runlog, and the only thing stopping the job is reaching the walltime. In these cases, there is no error message, which is a severe setback in determining the cause. The watchdog was essentially developed for cases like this. It was suspected that something in the code might cause this, but it is the suspicion of the authors that the problem might be twofold: there might be cases where something in the code causes this behaviour, and other cases where it is hardware-related. This suspicion is based on the highly random nature of this occurrence. There were many cases of such runs in February-March, and there are still some cases to this day. There were attempts to try different sets of modules, adding commands to the jobscript (MPI_LAUNCH_TIMEOUT), trying out with and without running postprocessing in the background, but nothing seems to completely solve the problem.

Table 1 contains a small playbook for previously encountered failures that have not been fully remedied yet.

Table 1: Known pipeline failure modes, their observable symptoms, and recommended recovery procedures.

Failure mode	**Observable symptoms**	**Diagnosis**	**Recovery procedure**	**Escalation**
GONG server unavailable or delayed	The magnetogram download produces HTTP errors, no newer GONG product is obtained, or `fitsfile.fits` remains unchanged for an extended period. The timestamp-advancement fallback may be activated repeatedly.	Inspect the magnetogram cron log and `fitsfile.source.txt`. Test the available GONG endpoints from the PFE on which the crontab is installed. Compare the source observation time with the modification time and FITS-header time of the local file.	Allow the timestamp fallback to maintain operational continuity. Test another GONG endpoint, or manually obtain the most recent MRZQS product when necessary. Confirm that the downloader resumes normal operation once the server becomes available.	Contact the NSO/GONG administrator if the server errors persist or all available endpoints remain inaccessible.
PFE does not execute the crontab	Cron-generated log files stop updating. Magnetogram downloads, transfers, job supervision, and CME-event checks may all stop even though previously submitted compute jobs continue to run. Results stop updating on the website.	Try to log in to the PFE on which the crontab was installed. This kind of error means that the whole PFE is offline, meaning login will noy work either.	Install the crontab on a functioning PFE. If at all possible, remove or disable the crontab on the original PFE before activating the replacement to avoid running duplicate instances of the pipeline scripts.	Contact NAS support if the PFE is unavailable or appears not to be executing scheduled commands.
PFE has no external network connectivity	Cron commands are invoked, but GONG downloads and transfers to Solstice fail. Local commands, including PBS job checks, may continue to operate normally.	Inspect the downloader and transfer logs for connection, DNS, timeout, or routing errors. Test access to the GONG server and Solstice directly from the same PFE that owns the crontab.	Move the crontab to a PFE with functioning external connectivity.	Contact NAS support if the PFE remains accessible but cannot reach external or University of Michigan systems.

Continued on the next page

Table 1: Known pipeline failure modes (continued)

Failure mode	**Observable symptoms**	**Diagnosis**	**Recovery procedure**	**Escalation**
Job is immediately held or returned to the queue	A submitted job is placed on hold or returned to the queue shortly after starting. Multiple job-start emails may be received for the same job, or several jobs may appear in a held state without manual intervention.	Inspect the job using `qstat`. Record the PBS job identifier, requested node type, start times, and any scheduler messages. This behavior may indicate one or more problematic compute nodes.	Delete and resubmit the affected job if appropriate. When repeated failures are associated with a particular node model, temporarily switch to an alternative supported node type.	Send NAS support the affected job identifiers, approximate failure times, requested node model, and relevant scheduler output.
Simulation terminates with a segmentation fault	The simulation ends unexpectedly and leaves a file named `core.<number>`. The run log may indicate a segmentation fault, often during an output-writing operation.	Open the core file with `gdb` using the exact SWMF executable that produced it and run `bt`. Preserve the core file, run log, PBS output, job identifier, and node information. Determine whether the backtrace points to an output-writing routine such as `ih_modwriteplot_mp_write_plot_`.	Restart the simulation from the latest valid checkpoint. If failures appear correlated with a node model, try a different supported model while retaining the failed run for investigation.	Contact NAS support when a hardware problem is suspected. Investigate more regarding the function in question.
PBS reports the job as running, but the simulation makes no progress	The job remains in the running state, but the newest `runlog` file and the output files in the `IO2` directories do not change. The job may remain inactive until it reaches its wall-time limit, without producing an explicit error.	Compare the modification time of the newest `runlog` file with the watchdog stale-time threshold. Inspect the PBS job state, node allocation, standard output, standard error, and the contents of the relevant `IO2` directories.	Allow `run_watchdog.sh` to terminate the unresponsive job and submit `job_CME_restart.sh`. If automatic restart is unavailable or the restart limit has been reached, cancel and restart the job manually from the latest valid restart state.	Escalate repeated cases with the job identifiers, node information, watchdog logs, run logs, and PBS output. Both numerical or configuration problems and hardware-related causes should be considered.